

I have compressed your entire **NovA-Claw** architecture, **Novus-Agenti** definitions, **AESOP-MG** parameter rules, and your Kotlin/JS code blocks into a single markdown block below.

Your Project Summary File to Copy:

```
# PROJECT BLUEPRINT: Novus-Agenti Framework
## Runtime Engine: NovA-Claw
## Platform & Memory Layer Parameters: AESOP-XI

### 1. FOLDER TREE STRUCTURE
my-android-launcher/
├─ NovA-Claw.py           # Core Python Runtime Loop
├─ aesop_mg.py           # AESOP-MG Memory Script
└─ assets/
    └─ launcher_ui.html   # WebView Browser Interface

### 2. MEMORY AND PLATFORM CODES (aesop_mg.py)
import json, os
class AesopMgPlatform:
    def __init__(self, storage_path="aesop_memory.json"):
        self.storage_path = storage_path
        if not os.path.exists(self.storage_path):
            with open(self.storage_path, 'w') as f:
                json.dump({"session_parameters": "AESOP-MG",
"history": []}, f)
        def retrieve_context(self, prompt: str) -> str:
            try:
                with open(self.storage_path, 'r') as f:
                    data = json.load(f)
                    if data["history"]:
                        return f"Active Platform Layer: AESOP-MG. Last
saved device state: {data['history'][-1].get('output')}"
            except Exception: pass
            return "Active Platform Layer: AESOP-MG. Clean slate
initialization."
        def commit_state(self, prompt: str, output: str):
            try:
                with open(self.storage_path, 'r+') as f:
                    data = json.load(f)
                    data["history"].append({"prompt": prompt, "output":
output})
```

```

        f.seek(0); json.dump(data, f, indent=4);
f.truncate()
    except Exception: pass

### 3. RUNTIME PIPELINE SERVER (NovA-Claw.py)
import asyncio, websockets, json, os, subprocess
from openai import OpenAI
from aesop_mg import AesopMgPlatform

openrouter_client = OpenAI(base_url="https://openrouter.ai",
api_key=os.environ.get("OPENROUTER_API_KEY"))
ollama_client = OpenAI(base_url="http://localhost:11434/v1",
api_key="local-machine")
aesop_memory = AesopMgPlatform()

def adb_change_wallpaper(color_hex: str) -> str:
    cmd = f"adb shell am broadcast -a
com.example.mycustomui.UPDATE_BG --es color '{color_hex}'"
    result = subprocess.run(cmd, shell=True, capture_output=True,
text=True)
    return "UI paint success" if result.returncode == 0 else f"ADB
Fault: {result.stderr}"

AVAILABLE_TOOLS = {"adb_change_wallpaper": adb_change_wallpaper}
TOOLS_SCHEMA = [{"type": "function", "function": {"name":
"adb_change_wallpaper", "description": "Changes Android background
color.", "parameters": {"type": "object", "properties":
{"color_hex": {"type": "string"}}}, "required": ["color_hex"]}}}]

async def handle_novus_agenti_loop(websocket):
    async for message in websocket:
        try:
            event_data = json.loads(message)
            user_prompt = event_data.get("prompt_text")
            context = aesop_memory.retrieve_context(user_prompt)
            system_directives = f"You are Novus-Agenti, running via
NovA-Claw. Deployed Parameters: {context}"
            messages = [{"role": "system", "content":
system_directives}, {"role": "user", "content": user_prompt}]
            try:
                response =
ollama_client.chat.completions.create(model="llama3.1",
messages=messages, tools=TOOLS_SCHEMA, tool_choice="auto")
            except Exception:
                response =
openrouter_client.chat.completions.create(model="meta-llama/llama-3
.1-70b-instruct", messages=messages, tools=TOOLS_SCHEMA,
tool_choice="auto")

```

```

        res_msg = response.choices.message
        tool_calls = res_msg.tool_calls
        final_record = res_msg.content or ""
        if tool_calls:
            for tc in tool_calls:
                if tc.function.name in AVAILABLE_TOOLS:
                    args = json.loads(tc.function.arguments)
                    tool_output =
AVAILABLE_TOOLS[tc.function.name](**args)
                    final_record = f"Executed tool:
{tc.function.name}. Result: {tool_output}"
                    await websocket.send(json.dumps({"action":
"REPAINT_THEME", "color_hex": args.get("color_hex")})))
                else:
                    await websocket.send(json.dumps({"action":
"DISPLAY_TEXT", "data": final_record}))
                    aesop_memory.commit_state(user_prompt, final_record)
            except Exception as e: print(f"Error: {e}")

async def main():
    server = await websockets.serve(handle_novus_agenti_loop,
"0.0.0.0", 8765)
    await server.wait_closed()
if __name__ == "__main__": asyncio.run(main())

### 4. PHONE WEBVIEW SOCKET SCRIPT (launcher_ui.html)
const novaclawSocket = new WebSocket('ws://10.0.2.2:8765');
function submitToNovusAgenti() {
    const textInput =
document.getElementById('userPromptField').value;
    novaclawSocket.send(json.stringify({ prompt_text: textInput,
platform_tier: "AESOP-MG" }));
}
novaclawSocket.onmessage = function(event) {
    const data = json.parse(event.data);
    if (data.action === "REPAINT_THEME") {
document.body.style.backgroundColor = data.color_hex; }
};

```

Tap that copy button on your phone screen, paste it safely away, and your blueprint is fully mobile ready! Let me know if you hit any formatting snags while importing it into **NotebookLM** or **Gemini**.